

Digital Design & Computer Arch.

Lecture 23b: Multi-Core Issues in Caching

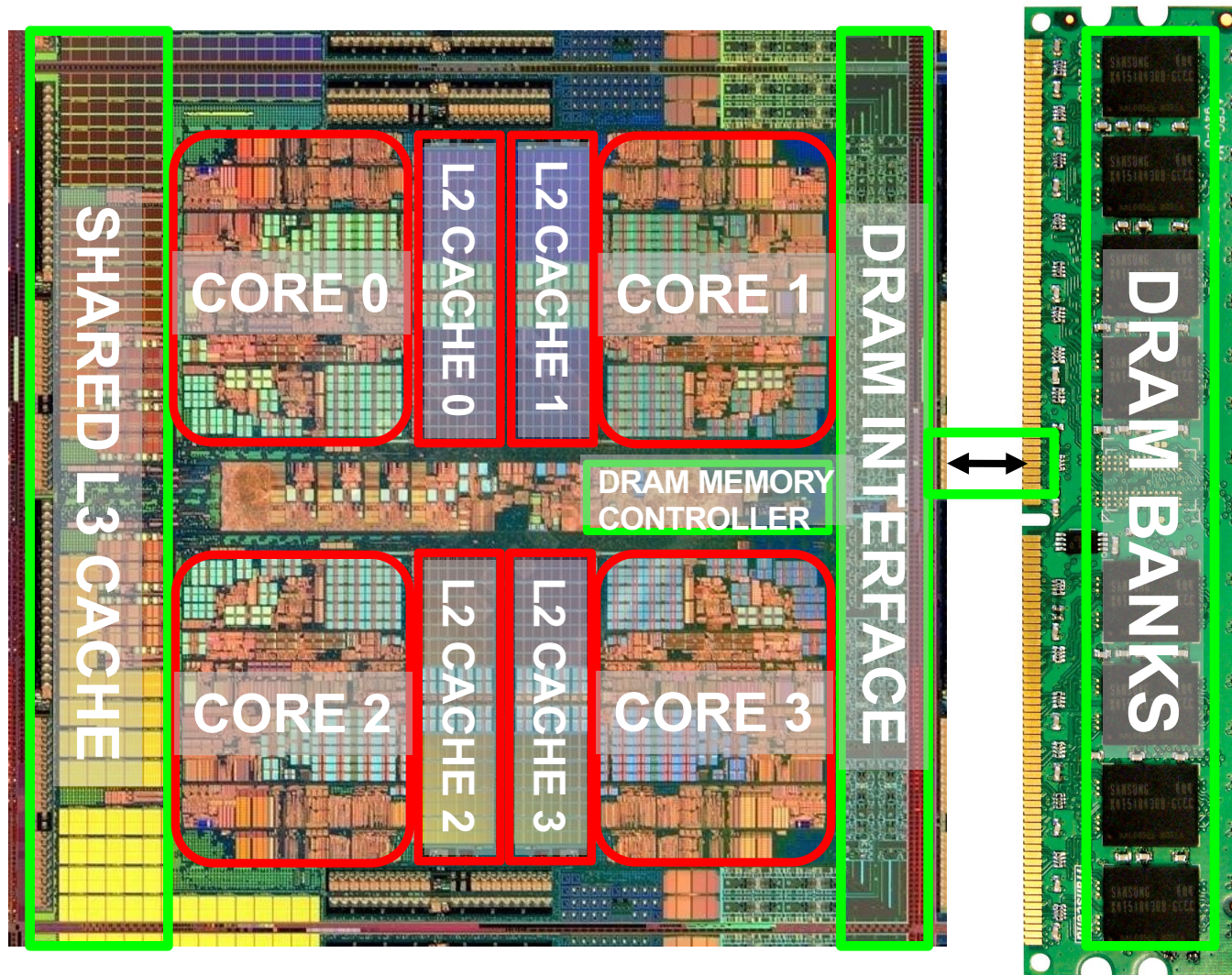
Prof. Onur Mutlu

ETH Zürich

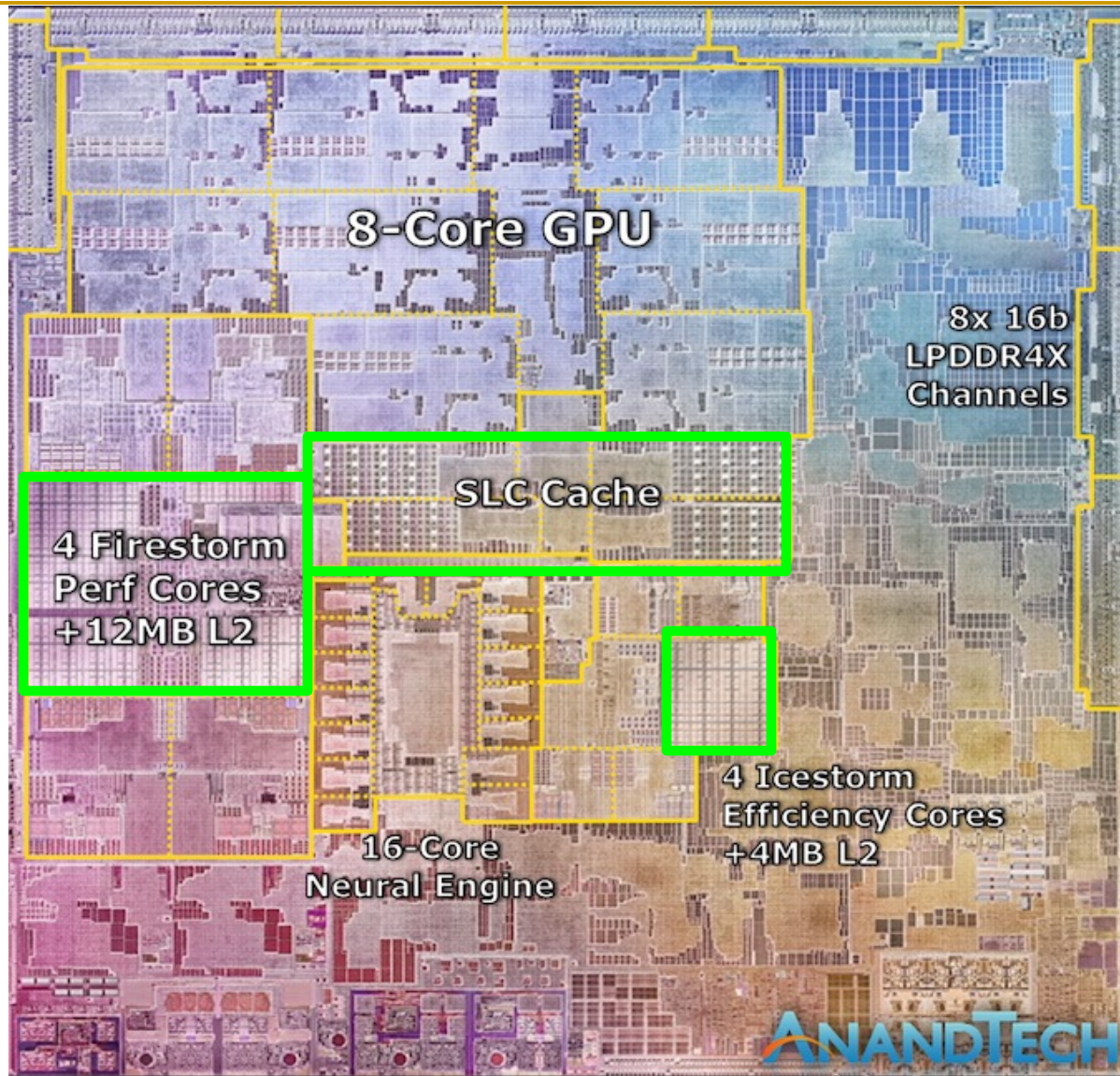
Spring 2025

22 May 2025

Caches in a Multi-Core System



Caches in a Multi-Core System



Apple M1,
2021

Caches in a Multi-Core System

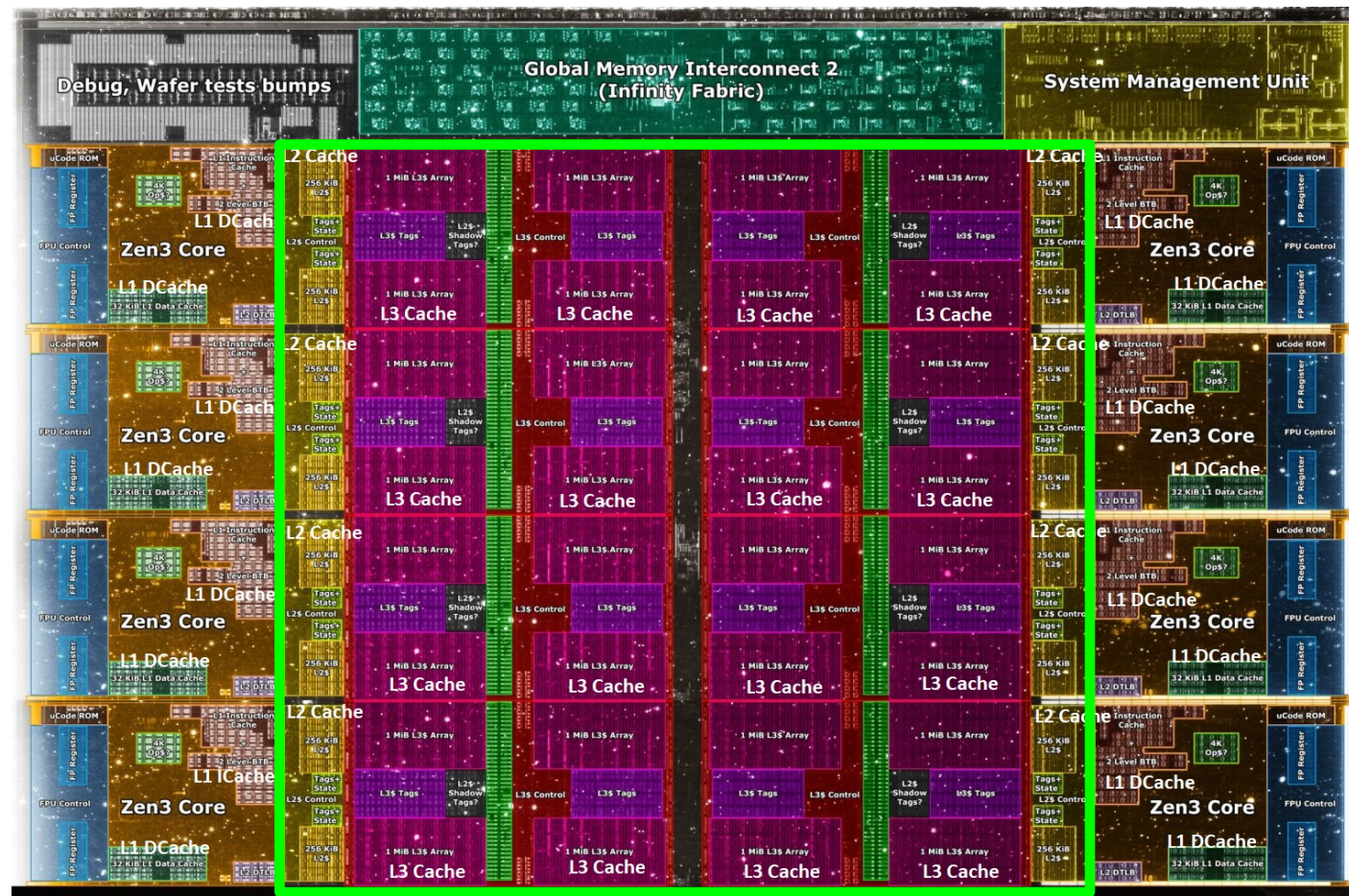


10nm ESF=Intel 7 Alder Lake die shot (~209mm²) from Intel: <https://www.intel.com/content/www/us/en/newsroom/news/12th-gen-core-processors.html>

Die shot interpretation by Locuza, October 2021

Intel Alder Lake,
2021

Caches in a Multi-Core System



Core Count:

8 cores/16 threads

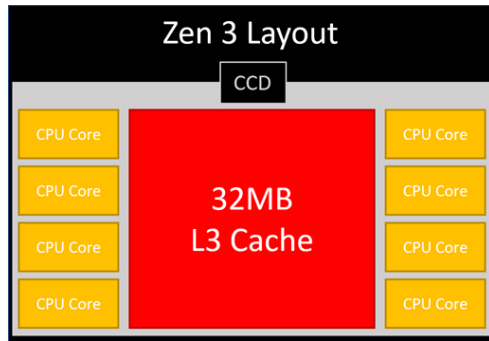
L1 Caches:
32 KB per core

L2 Caches:
512 KB per core

L3 Cache:
32 MB shared

AMD Ryzen 5000, 2020

Caches in a Multi-Core System

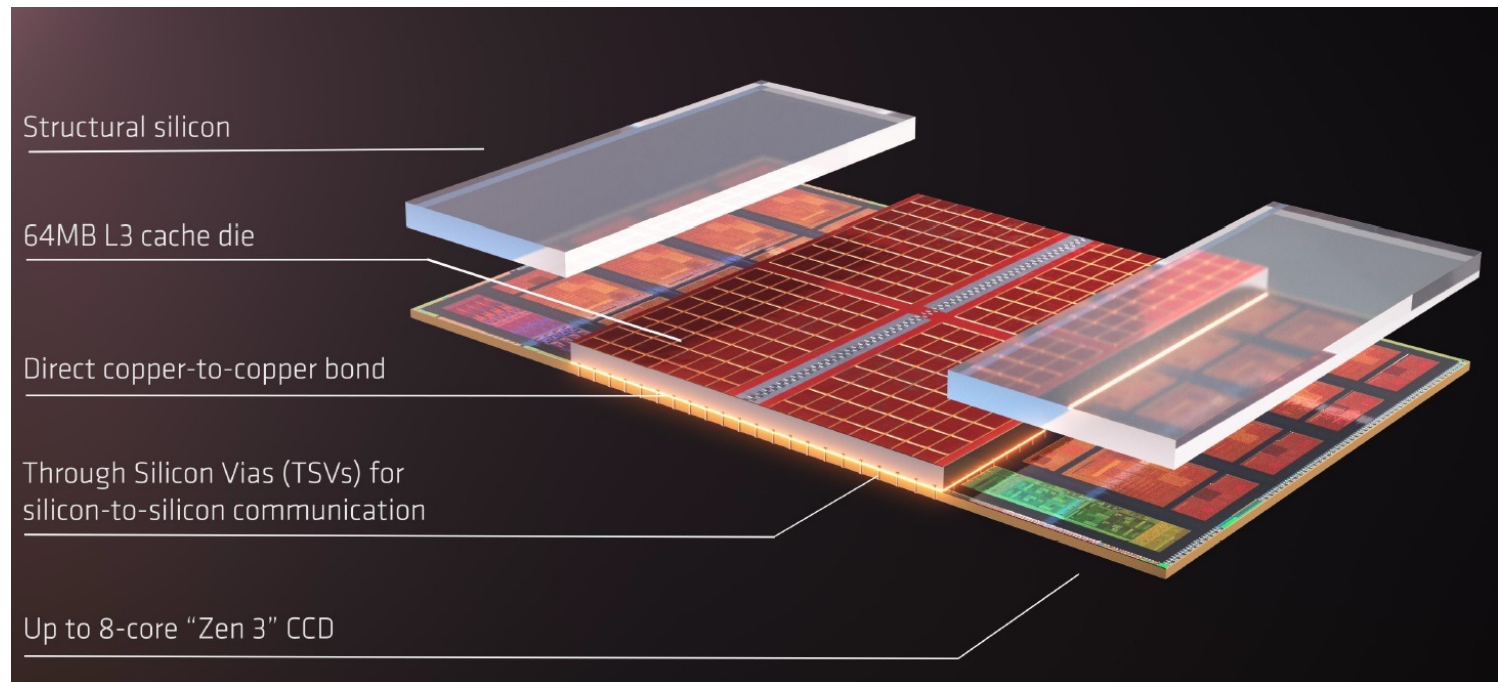


<https://community.microcenter.com/discussion/5134/comparing-zen-3-to-zen-2>

AMD increases the L3 size of their 8-core Zen 3 processors from 32 MB to 96 MB

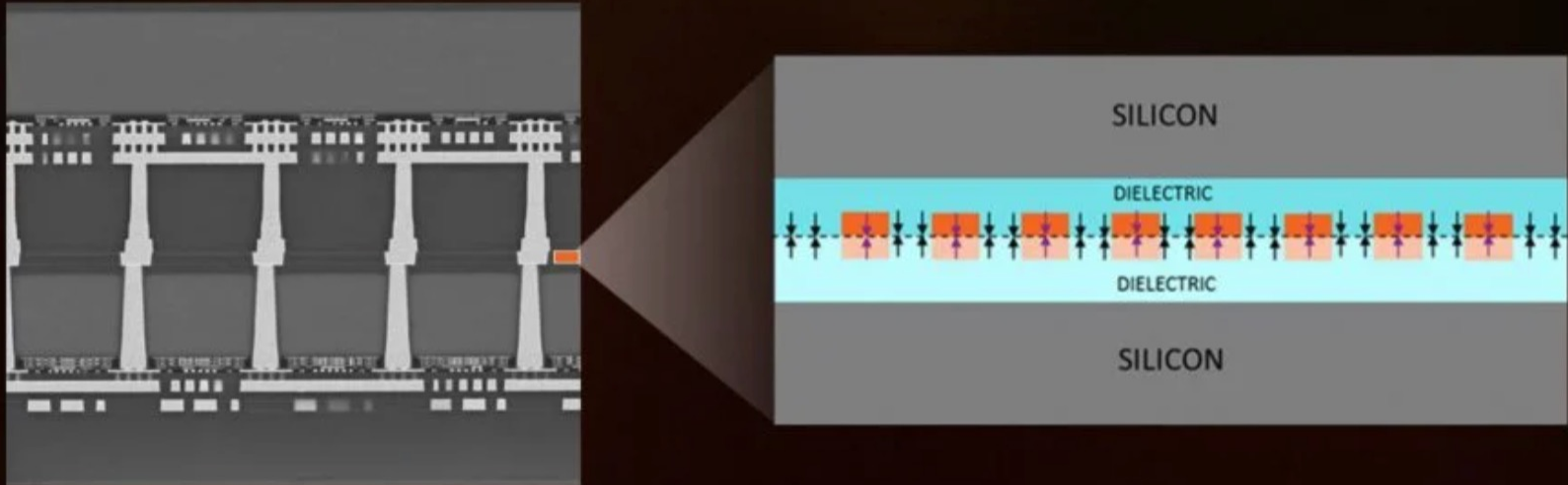
Additional 64 MB L3 cache die
stacked on top of the processor die

- Connected using Through Silicon Vias (TSVs)
- Total of 96 MB L3 cache



3D Stacking Technology: Example

AMD 3D V-CACHE™ HYBRID BONDING

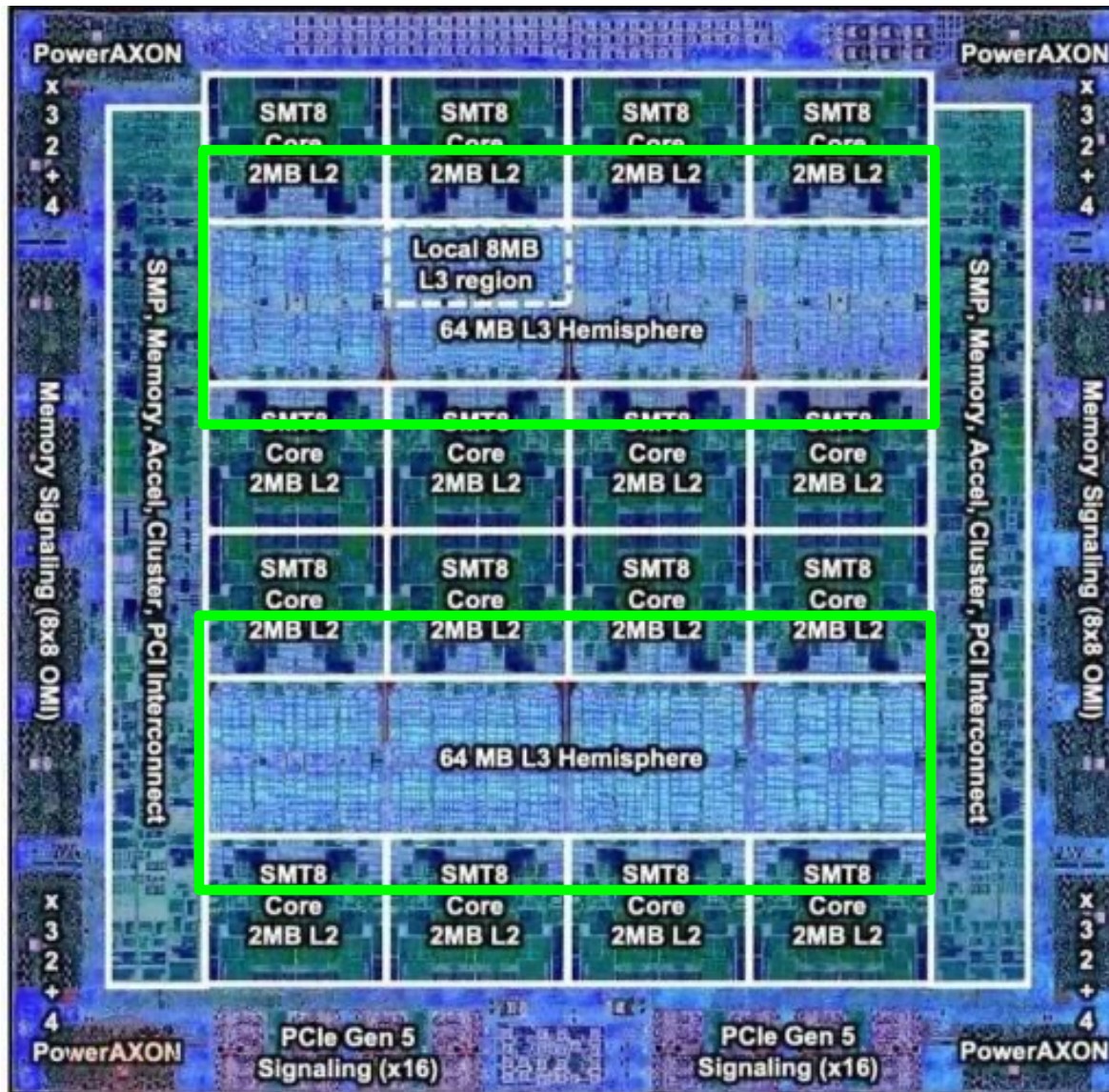


Direct Copper –Copper Bonding

AMD Ryzen 7 5800X3D: The 3D V-Cache in detail (4)

Source: AMD

Caches in a Multi-Core System



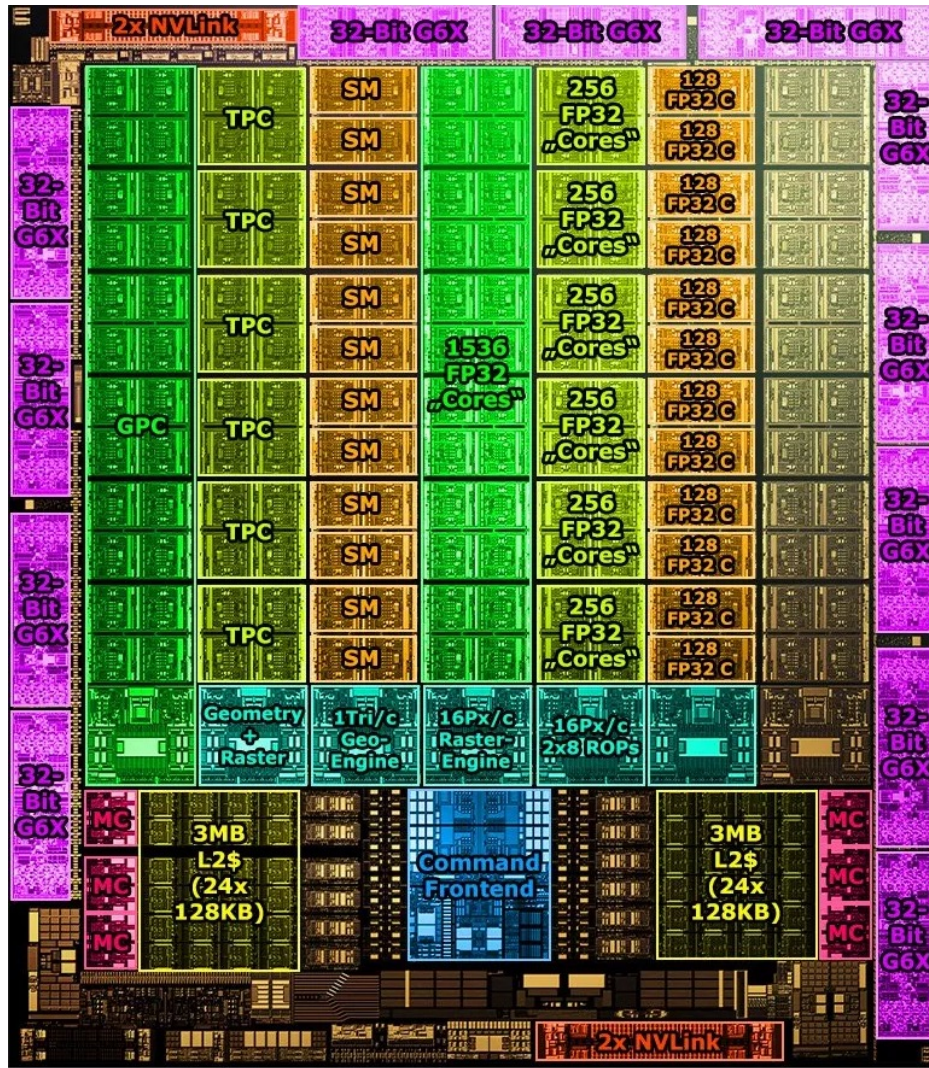
IBM POWER10,
2020

Cores:
15-16 cores,
8 threads/core

L2 Caches:
2 MB per core

L3 Cache:
120 MB shared

Caches in a Multi-Core System



Nvidia Ampere, 2020

Cores:

128 Streaming Multiprocessors

L1 Cache or Scratchpad:

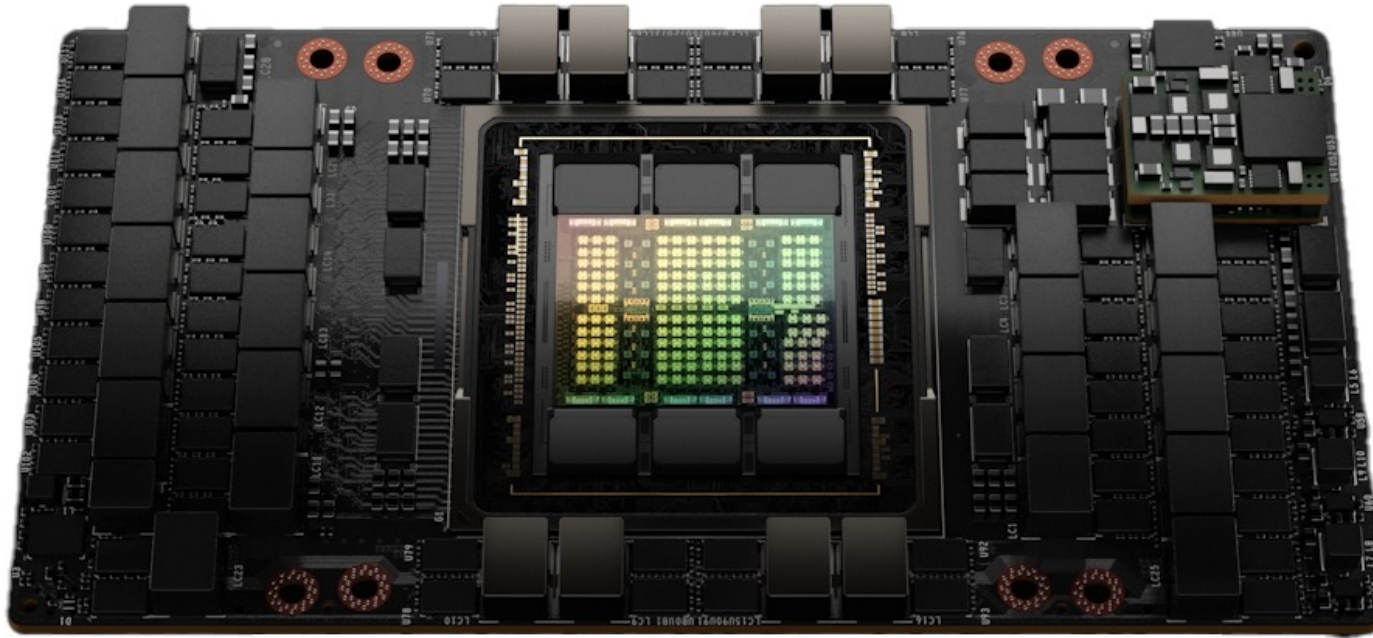
192KB per SM

Can be used as L1 Cache
and/or Scratchpad

L2 Cache:

40 MB shared

Caches in a Multi-Core System



Nvidia Hopper,
2022

Cores:
144 Streaming
Multiprocessors

**L1 Cache or
Scratchpad:**
256KB per SM
Can be used as L1 Cache
and/or Scratchpad

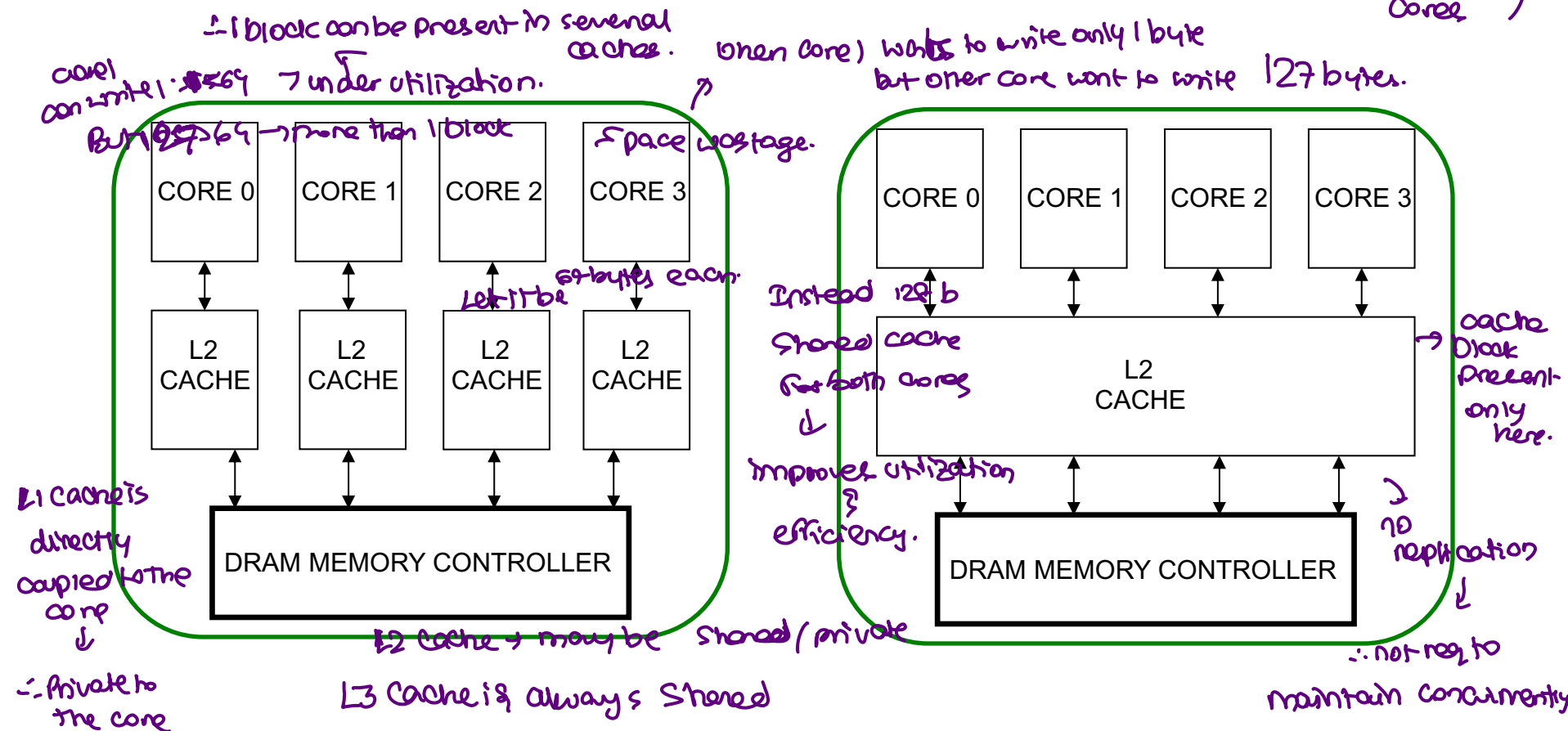
L2 Cache:
60 MB shared

Caches in Multi-Core Systems

- Cache efficiency becomes even more important in a multi-core/multi-threaded system (To maintain concurrency b/w several threads)
 - Memory bandwidth is at premium (We don't want to go to main memory we want hit to occur at cache itself)
 - Cache space is a limited resource across cores/threads
- How do we design the caches in a multi-core system?
- Many decisions and questions
 - Shared vs. private caches (across cores/threads)
 - How to maximize performance of the entire system?
 - How to provide QoS & predictable perf. to different threads in a shared cache?
 - Should cache management algorithms be aware of threads?
 - How should space be allocated to threads in a shared cache?
 - Should we store data in compressed format in some caches?
 - How do we do better reuse prediction & management in caches?

Private vs. Shared Caches

- **Private** cache: Cache belongs to one core (a shared block can be in multiple caches)
- **Shared** cache: Cache is shared by multiple cores (Either all cores or subset of cores)



∴ The concept.

Resource Sharing Concept and Advantages

- Idea: Instead of dedicating a hardware resource to a hardware context, allow multiple contexts to use it
 - Example resources: functional units, pipeline, caches, buses, memory, interconnects, storage
- Why?
 - What is the benefit of resource sharing.
 - Cache → Instruction Cache & Data Cache.
- + Resource sharing improves utilization/efficiency → throughput
 - ① □ (When a resource is left idle by one thread, another thread can use it) (no need to replicate shared data) ②.
- + Reduces communication latency
 - For example, data shared between multiple threads can be kept in the same cache in multithreaded processors
- + Compatible with the shared memory programming model
 - Threads communicate with each other using load & store instr's.
 - (Memory shared by several threads)
 - 1 inst writing at a addr by 1 thread can be seen by other threads.

Resource Sharing Disadvantages

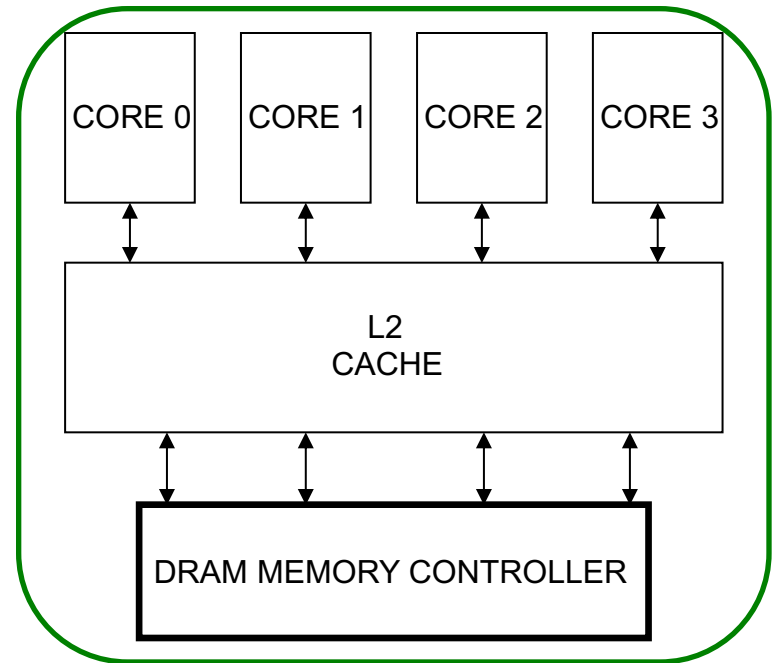
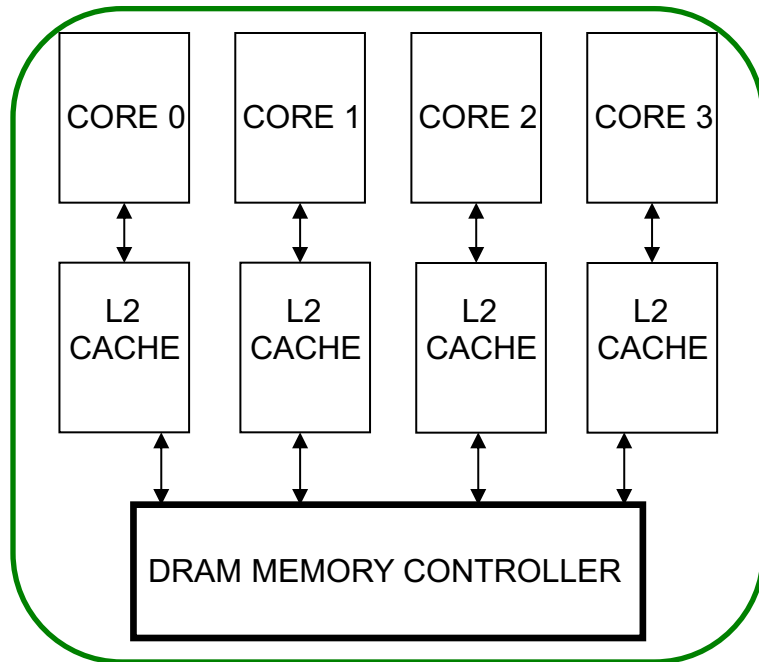
To prevent this we go with resource replication.

- Resource sharing results in **contention for resources**
 - When the resource is not idle, another thread cannot use it
 - If space is occupied by one thread, another thread needs to re-occupy it
- Sometimes reduces each or some thread's performance
 - Thread performance can be worse than when it is run alone
- **Eliminates performance isolation** → inconsistent performance across runs (i.e. we program our code in order to fit it in the cache ∴ better perf when run alone but now if we run this program in parallel with other program B, B may evict all blocks related to program A ∴ the perf with program A alone is now lost due to program B sharing the cache.)
- Thread performance depends on co-executing threads
- Uncontrolled (free-for-all) sharing **degrades quality of service**
- Causes unfairness, starvation

Need to efficiently and fairly utilize shared resources

Private vs. Shared Caches

- **Private** cache: Cache belongs to one core (a shared block can be in multiple caches)
- **Shared** cache: Cache is shared by multiple cores



Shared Caches Between Cores

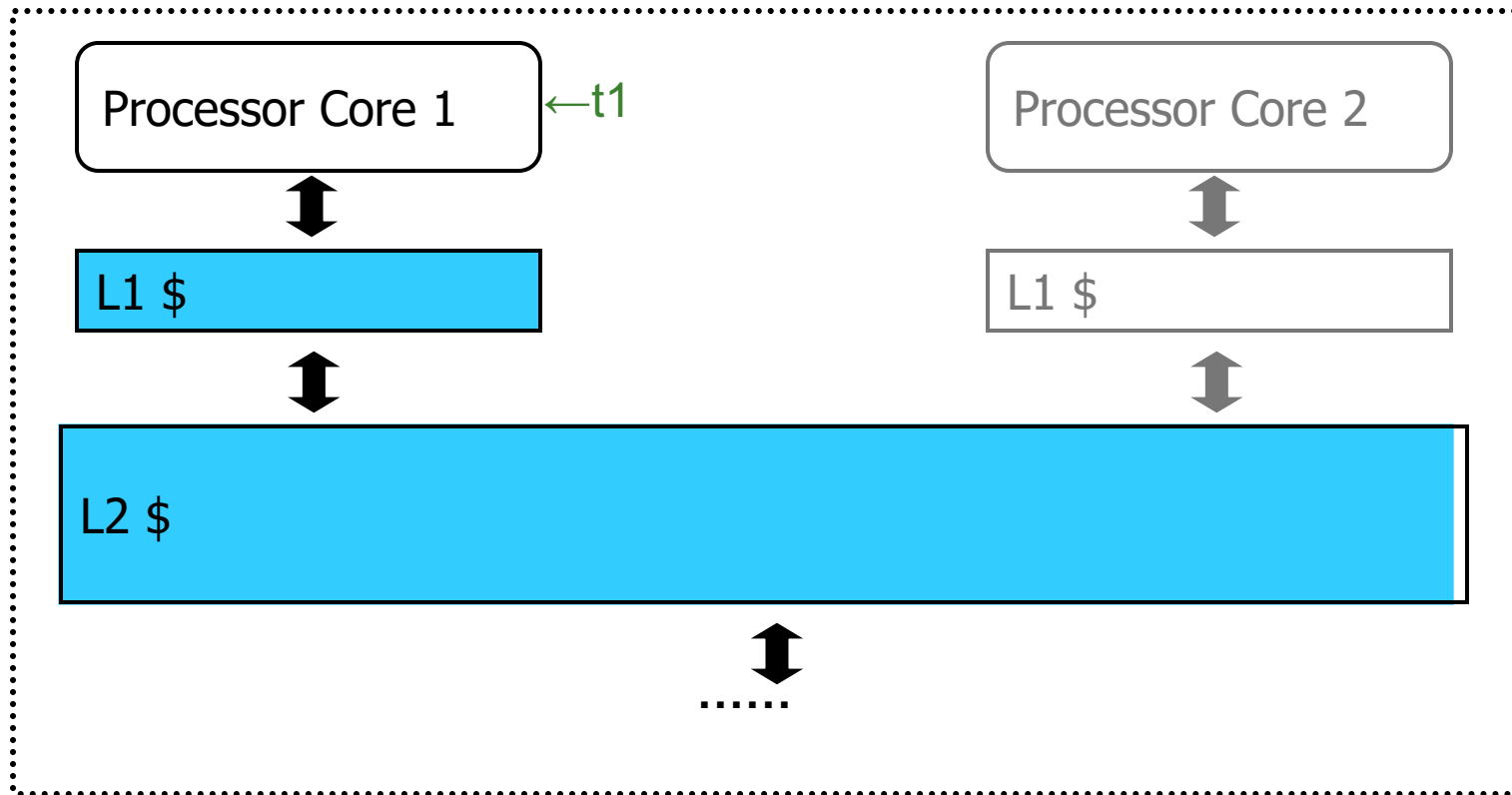
■ Advantages:

- High effective capacity
- Dynamic partitioning of available cache space
 - No fragmentation due to static partitioning (A)
 - If one core does not utilize some space, another core can
- Easier to maintain coherence (a cache block is in a single location)

■ Disadvantages

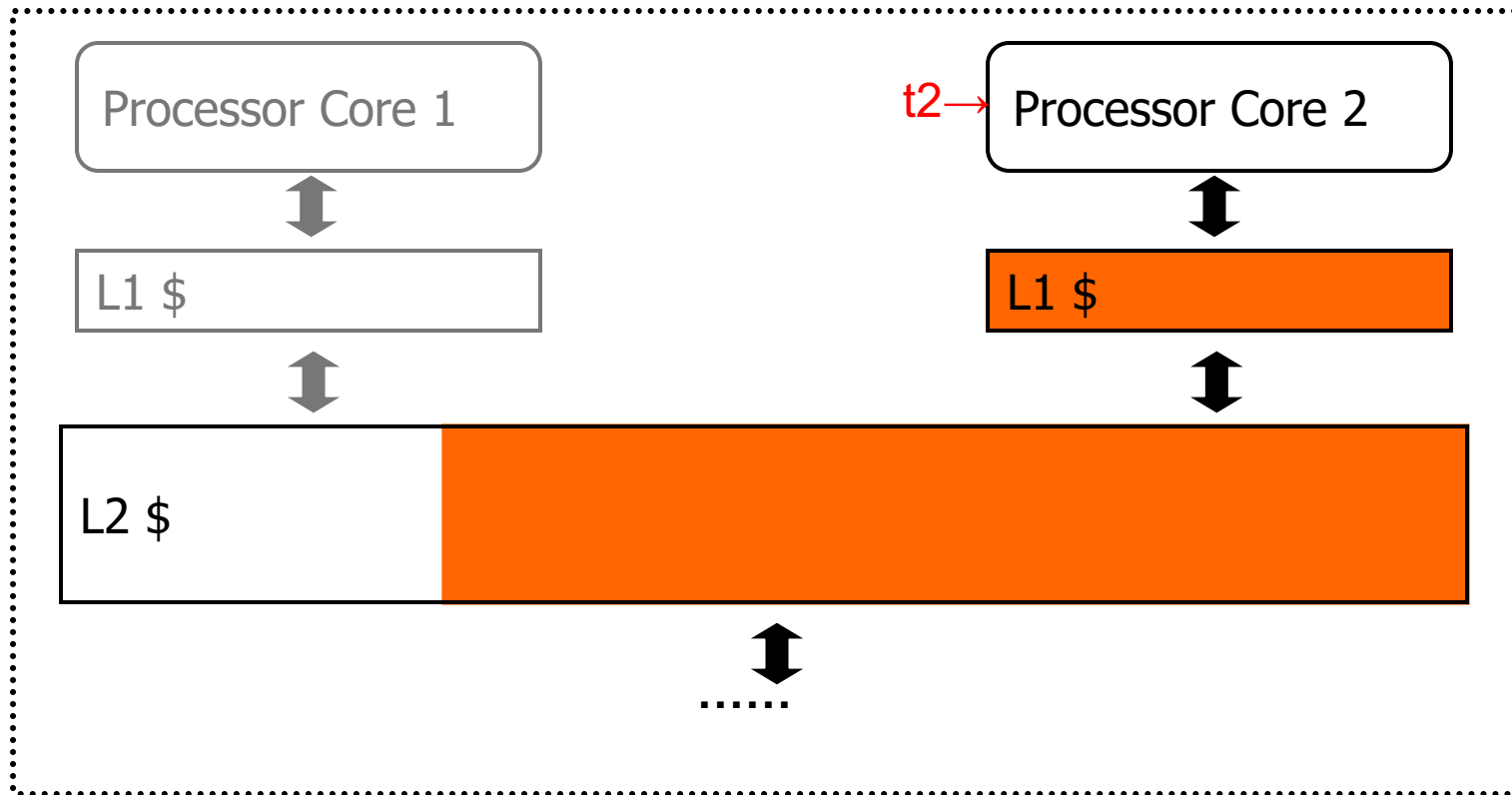
- Slower access (cache is not tightly coupled with the core)
- Cores incur conflict misses due to other cores' accesses
 - Misses due to inter-core interference
 - Some cores can destroy the hit rate of other cores
- Guaranteeing a minimum level of service (or fairness) to each core is harder (how much space, how much bandwidth?)

Example: One Problem with Shared Caches



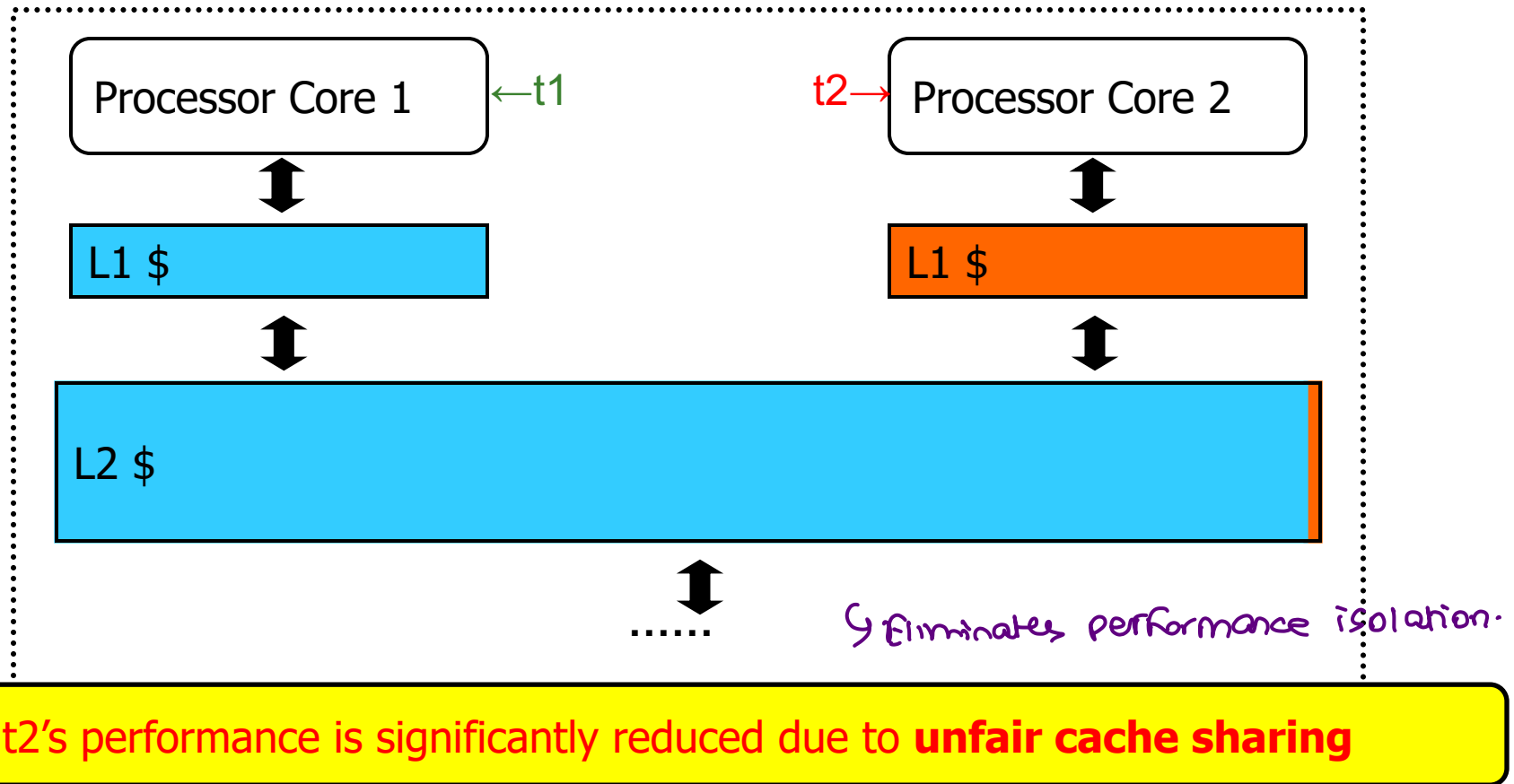
Kim et al., “Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture,” PACT 2004.

Example: One Problem with Shared Caches



Kim et al., “Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture,” PACT 2004.

Example: One Problem with Shared Caches



Kim et al., "Fair Cache Sharing and Partitioning in a Chip Multiprocessor Architecture," PACT 2004.

Resource Sharing vs. Partitioning

- Sharing improves resource utilization
 - Better utilization of space
- Partitioning provides performance isolation (predictable performance)
 - Dedicated space
- Can we get the benefits of both?
- Idea: Design shared resources such that they are efficiently utilized, controllable and partitionable
 - No wasted resource + QoS mechanisms for threads

Lectures on Multi-Core Cache Management

Computer Architecture

Lecture 15:

Multi-Core Cache Management

Prof. Onur Mutlu
ETH Zürich
Fall 2017
15 November 2017



0:35 / 2:33:03



Computer Architecture - Lecture 15: Multi-Core Cache Management (ETH Zürich, Fall 2017)

934 views • Nov 17, 2017

👍 13 💬 0 ➦ SHARE ⚙️ SAVE ...

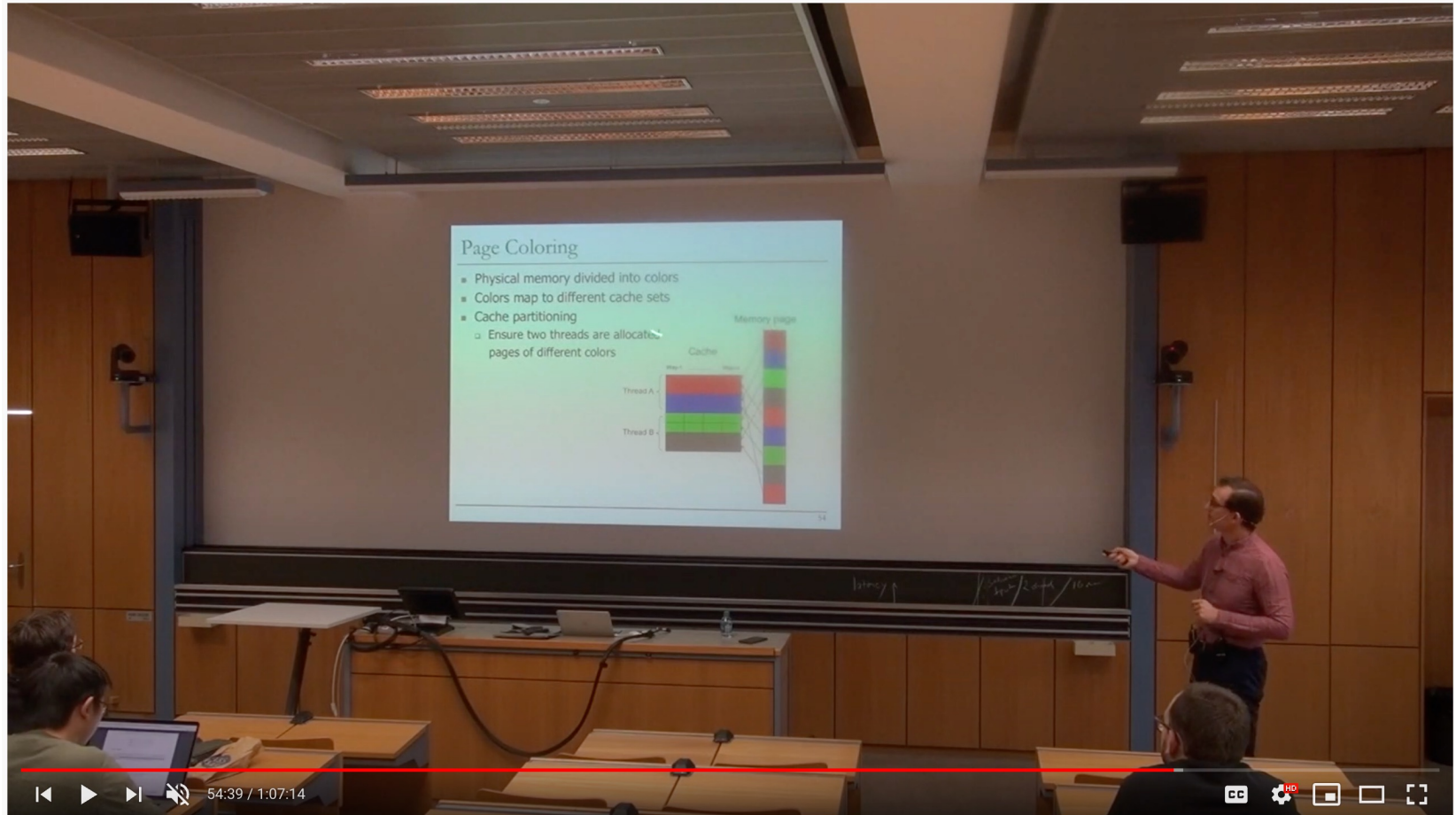


Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

Lectures on Multi-Core Cache Management



ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 18b: Multi-Core Cache Management (ETH Zürich, Fall 2018)

744 views • Nov 23, 2018

12 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

https://www.youtube.com/watch?v=c9FhGRB3HoA&list=PL5Q2soXY2Zi9JXe3ywQMhyIk_d5dl-TM7&index=29

Lectures on Multi-Core Cache Management

Approaches to Reuse Prediction

Use program counter or memory region information.

1. Group Blocks 2. Learn group behavior 3. Predict reuse

PC 1 PC 2 PC 1 PC 2 PC 1 PC 2

A B S T A B S T C → C U → U

1. Same group ↗ same reuse behavior
2. No control over number of high-reuse blocks

77

ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 19a: Multi-Core Cache Management II (ETH Zürich, Fall 2018)

293 views • Dec 2, 2018

6 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

https://www.youtube.com/watch?v=Siz86_PD4w&list=PL5Q2soXY2Zi9JXe3ywQMhyIk_d5dl-TM7&index=30

Lectures on Multi-Core Cache Management

- **Computer Architecture, Fall 2018, Lecture 18b**
 - Multi-Core Cache Management (ETH, Fall 2018)
 - https://www.youtube.com/watch?v=c9FhGRB3HoA&list=PL5Q2soXY2Zi9JXe3ywQMhylk_d5dI-TM7&index=29

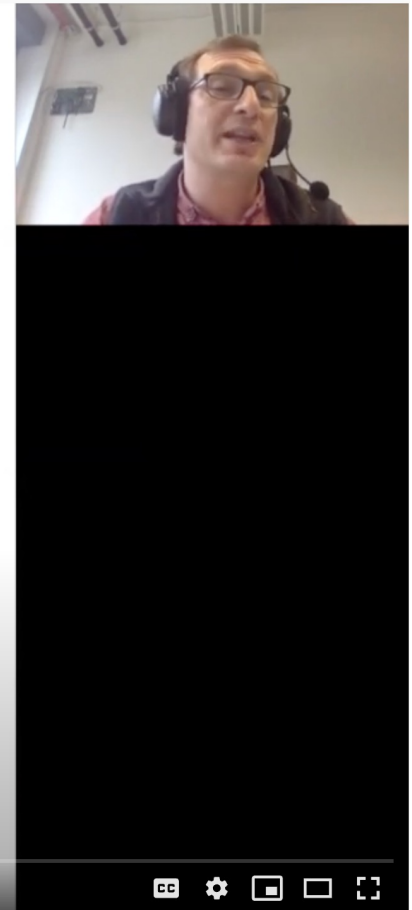
- **Computer Architecture, Fall 2018, Lecture 19a**
 - Multi-Core Cache Management II (ETH, Fall 2018)
 - https://www.youtube.com/watch?v=Siz86__PD4w&list=PL5Q2soXY2Zi9JXe3ywQMhylk_d5dI-TM7&index=30

- **Computer Architecture, Fall 2017, Lecture 15**
 - Multi-Core Cache Management (ETH, Fall 2017)
 - https://www.youtube.com/watch?v=7_Tqlw8gxOU&list=PL5Q2soXY2Zi9OhoVQBXyFIZywZXCPI4M_&index=17

Lectures on Memory Resource Management

QoS-Aware Memory Systems: Challenges

- How do we **reduce inter-thread interference**?
 - ❑ Improve system performance and core utilization
 - ❑ Reduce request serialization and core starvation
- How do we **control inter-thread interference**?
 - ❑ Provide mechanisms to enable system software to enforce QoS policies
 - ❑ While providing high system performance
- How do we **make the memory system configurable/flexible**?
 - ❑ Enable flexible mechanisms that can achieve many goals
 - Provide fairness or throughput when needed
 - Satisfy performance guarantees when needed



ETH ZÜRICH HAUPTGEBÄUDE

Computer Architecture - Lecture 11b: Memory Interference and QoS (ETH Zürich, Fall 2020)

735 views • Oct 31, 2020

14 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS

EDIT VIDEO

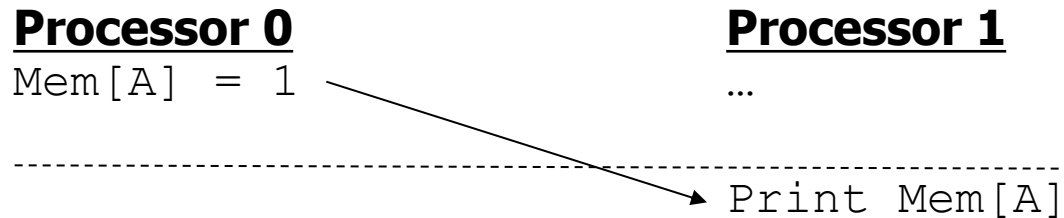
Lectures on Memory Resource Management

- **Computer Architecture, Fall 2020, Lecture 11a**
 - Memory Controllers (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=TeG773OgiMQ&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=20>
- **Computer Architecture, Fall 2020, Lecture 11b**
 - Memory Interference and QoS (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=0nnI807nCkc&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=21>
- **Computer Architecture, Fall 2020, Lecture 13**
 - Memory Interference and QoS II (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=Axye9VqQT7w&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=26>
- **Computer Architecture, Fall 2020, Lecture 2a**
 - Memory Performance Attacks (ETH, Fall 2020)
 - <https://www.youtube.com/watch?v=VJzZbwgBfy8&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=2>

Cache Coherence

Shared Memory Model

- Threads in parallel programs communicate through *shared memory*
- Thread 0 writes to an address, followed by Thread 1 reading
 - This implies communication between the two



- Each read should receive the value last written by any processor
 - This requires synchronization between threads (i.e., what does last written mean?)
- What if `Mem[A]` is cached (at either processor)?

Cache Coherence

- Basic question: If multiple processors cache the same block, how do they ensure they all see a consistent state?

i.e. both Cache 1 ?

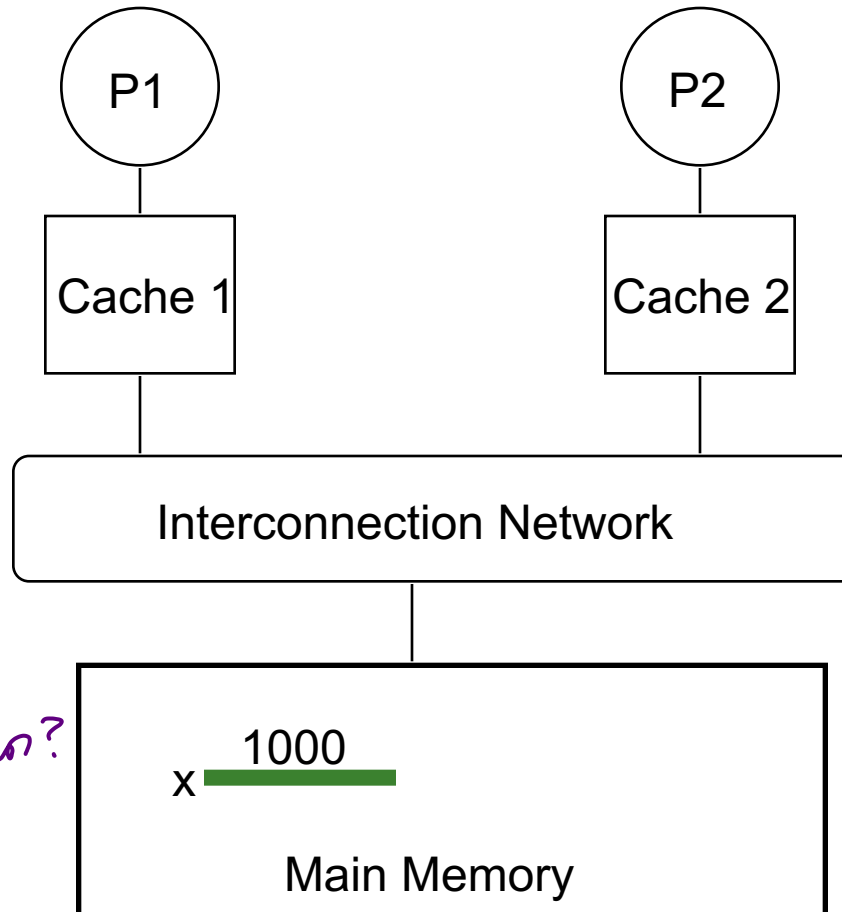
Cache 2 have

Some block how do
they ensure that
both the processors
see the latest data
in both the blocks.

-> How does

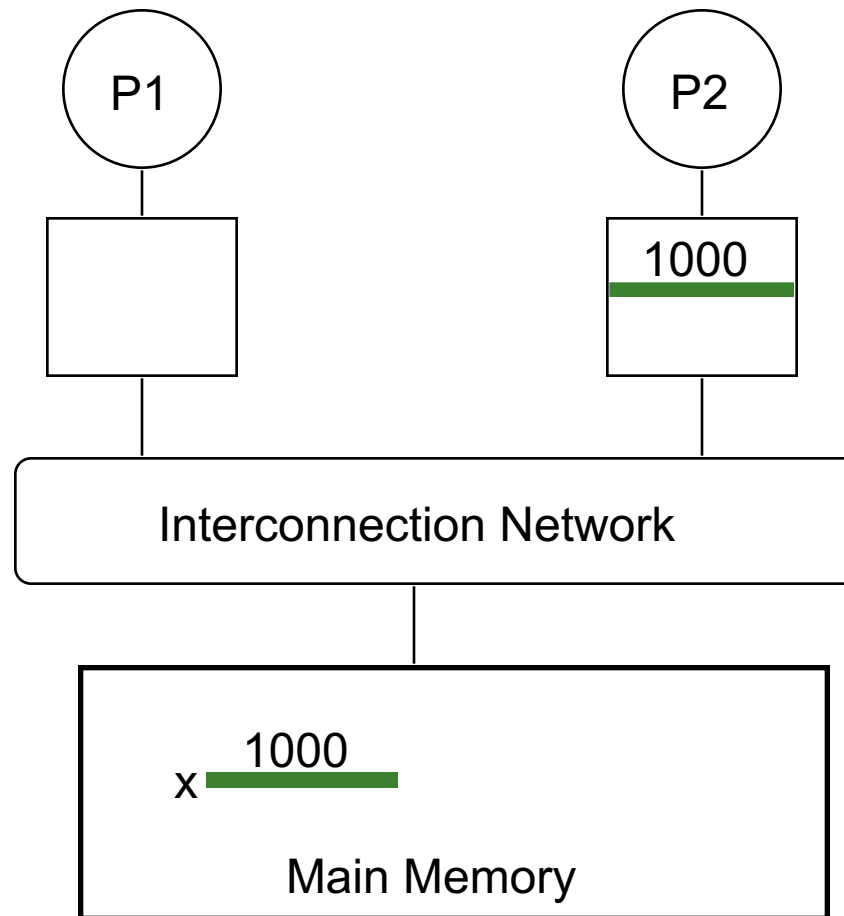
consistency

maintain happen?



-> to communicate
to main memory

The Cache Coherence Problem



ld r2, x

x gets cached in
cache 2 → P2 gets the
data from main
memory

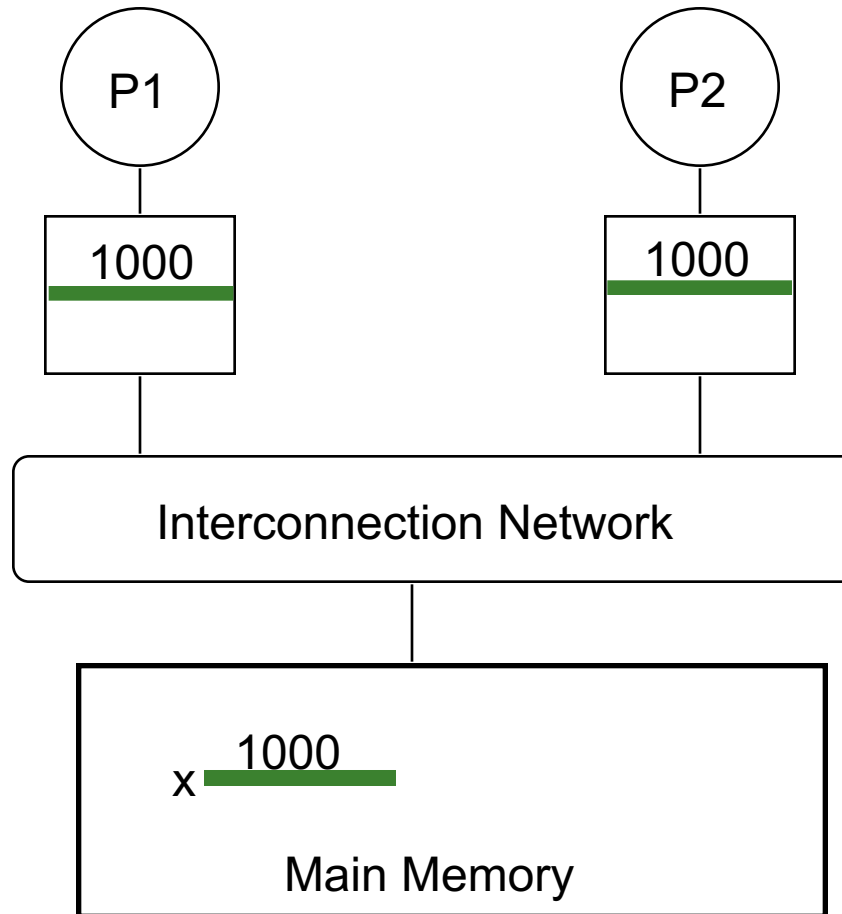
↓
Then load the
data into
register r2.

The Cache Coherence Problem

ld r2, x

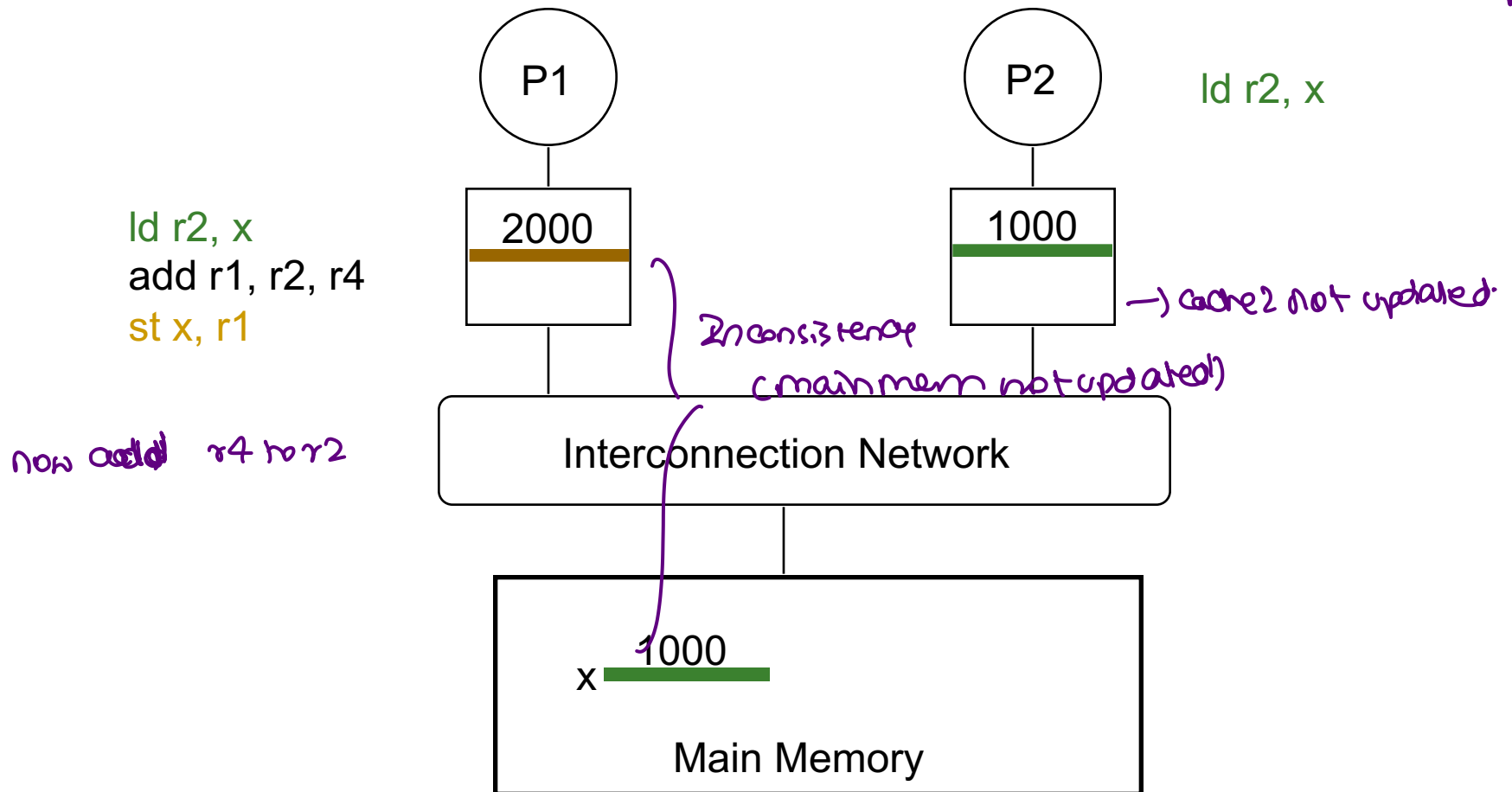
ld r2, x

- ① Get x from MM
- ② Store x in cache1
- ③ Load the value of x in register r2.

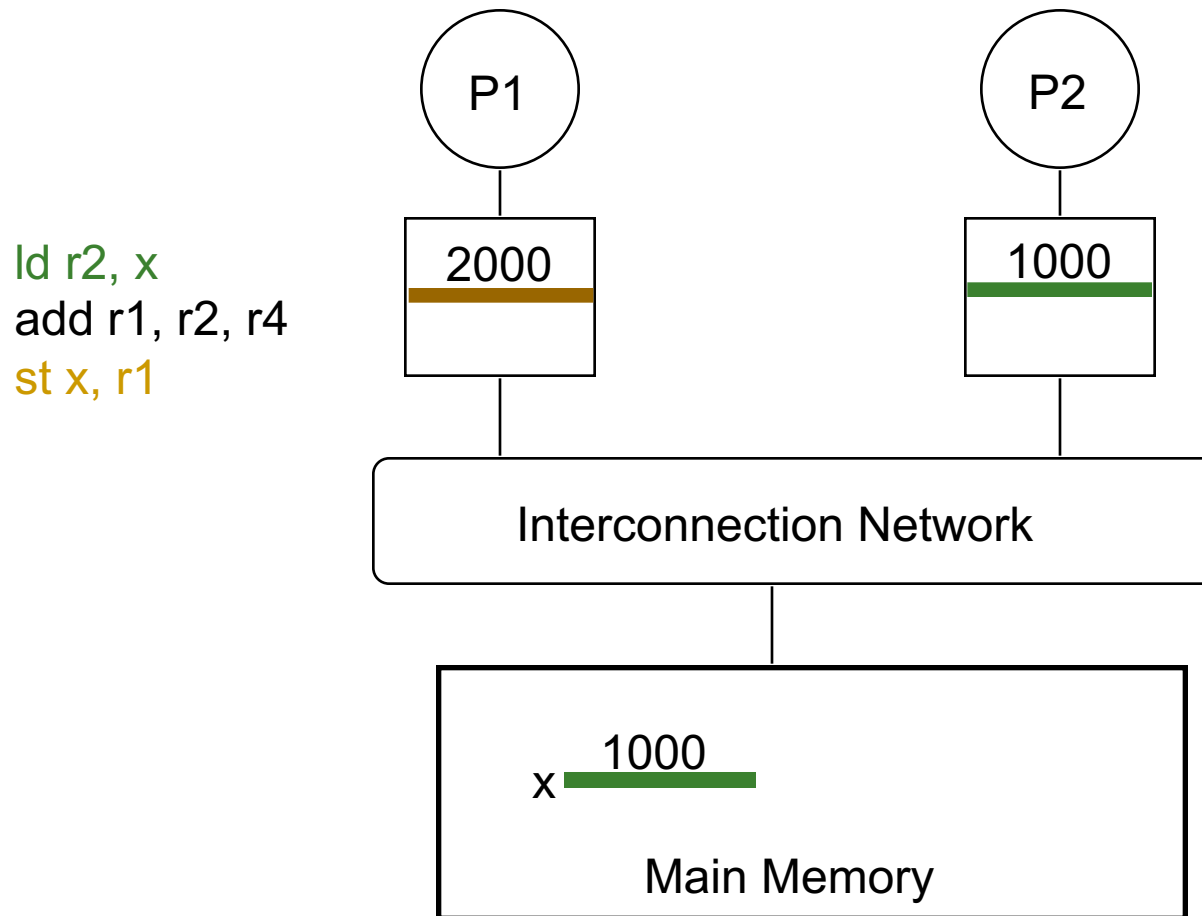


The Cache Coherence Problem

Let us assume it's write back cache → ∴ Any update made we write only in the cache not in mem.



The Cache Coherence Problem



∴ value got updated before we are reading

ld r2, x

↖

Should NOT load 1000

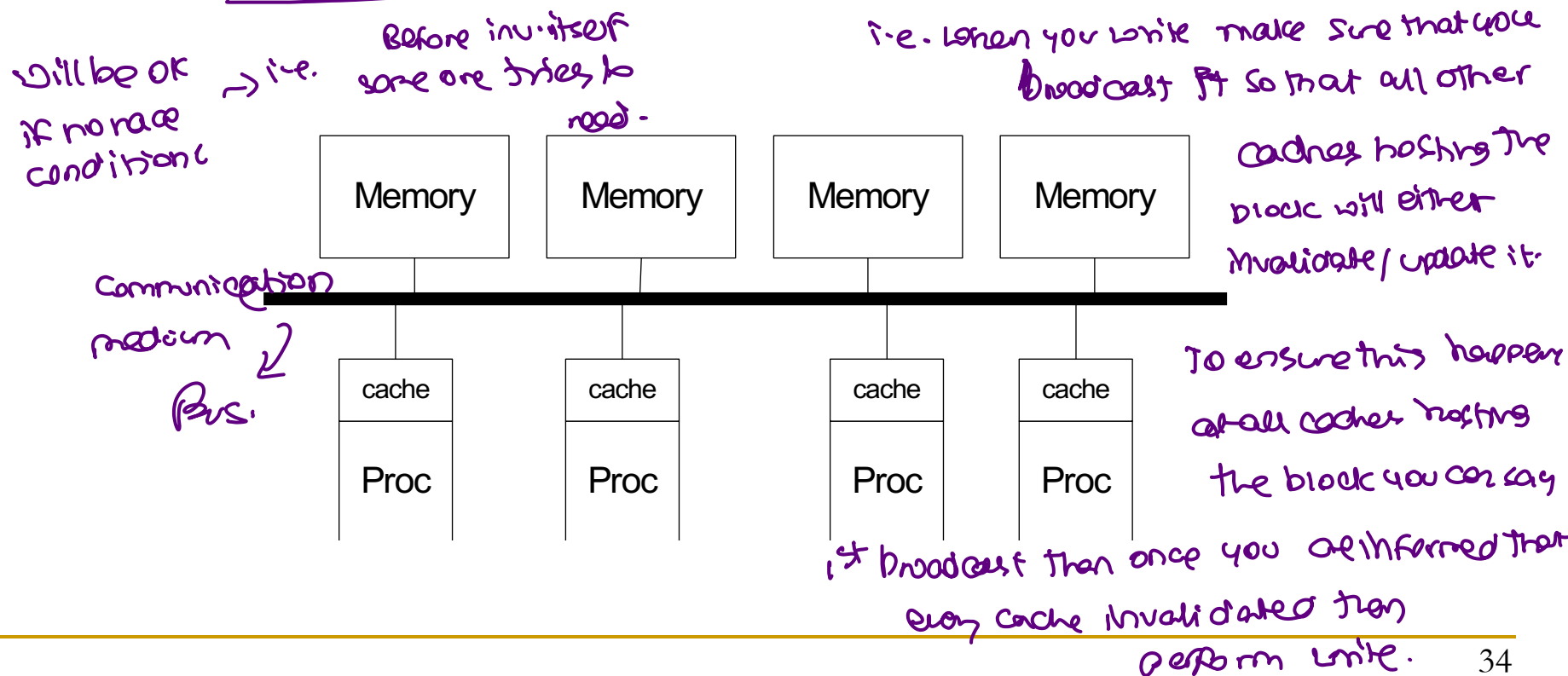
ld r5, x

↓

As per sequentially Semantics of the model.

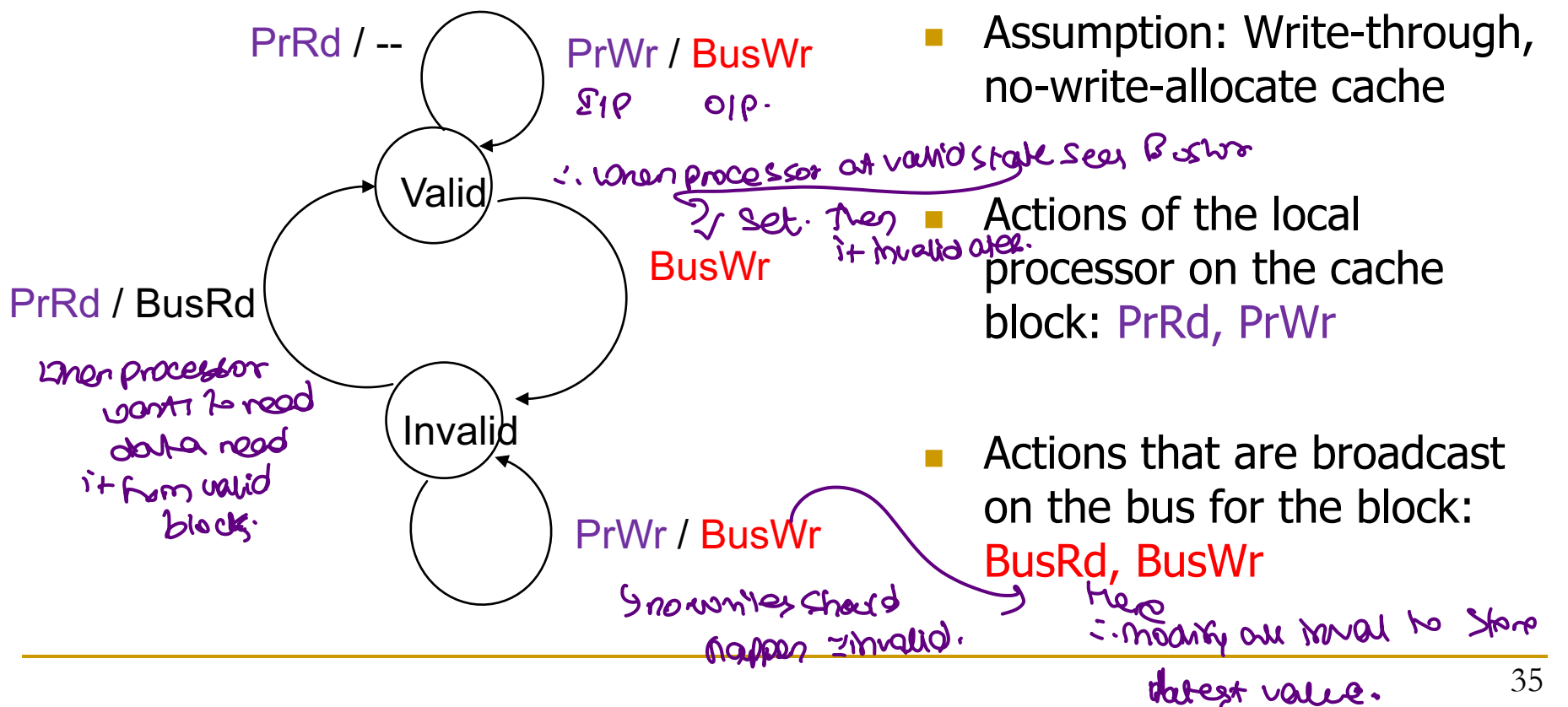
Broadcast-Based Hardware Cache Coherence

- Basic idea: Idea - 1
 - A processor/cache broadcasts its write/update to a cache block to all other processors
 - Another processor/cache that has the block either invalidates or updates its local copy of the block



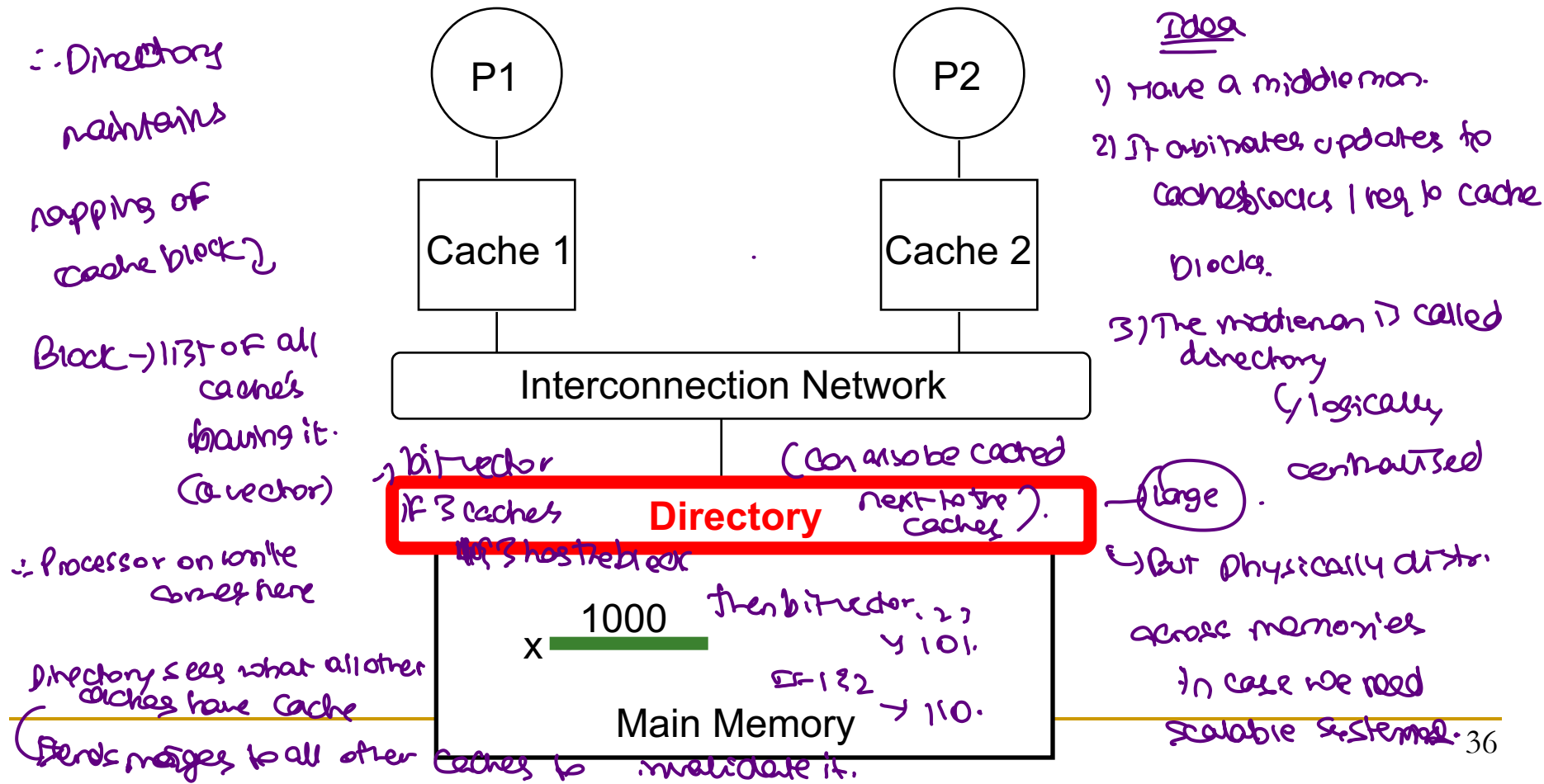
A Very Simple Coherence Scheme (VI)

- Idea: All caches “snoop” (observe) each other’s write/read operations. If a processor writes to a block, all others invalidate the block.
- A simple protocol:



Directory-Based Cache Coherence

- Idea: A central directory keeps track of which caches contain every possible cache block. Every cache consults this directory to ensure data is kept coherent.



Directory-Based Cache Coherence

- Idea: A central directory keeps track of which caches contain every possible cache block. Every cache consults this directory to ensure data is kept coherent. $(P_0, P_1, P_2, \dots, P_{P-1})$

P bits $\rightarrow$ for P no. of processors

- An example mechanism:

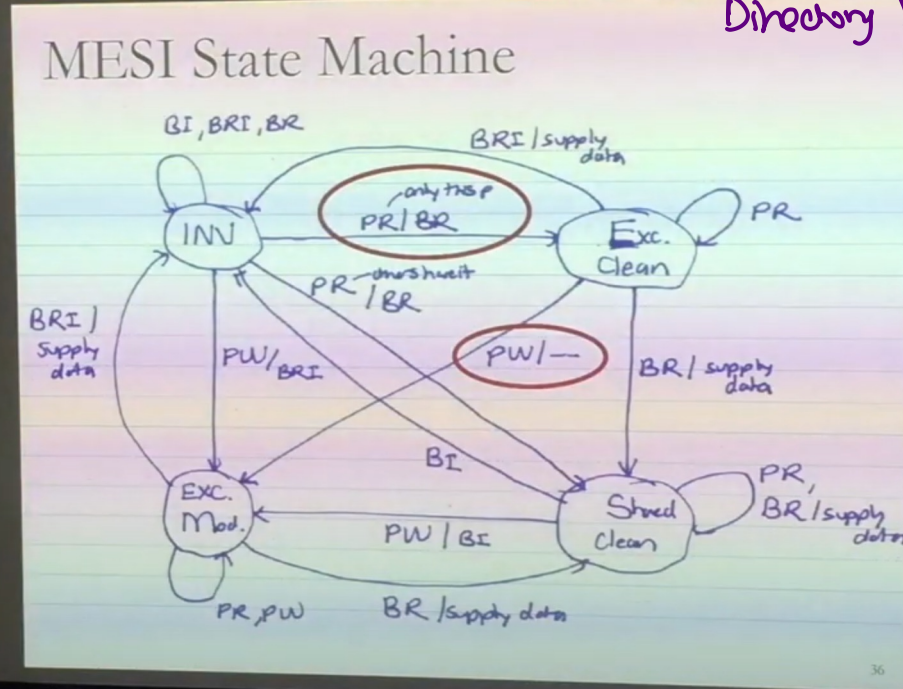
1 bit $\rightarrow$ exclusive bit

- For each cache block in memory, store $P+1$ bits in directory
 - One bit for each cache, indicating whether the block is in cache
 - Exclusive bit: indicates that a cache has the only copy of the block and can update it without notifying others
- On a read: set the cache's bit in the directory entry for the block and arrange the supply of the block into the cache
 (Retrieve from main or cache that has latest valid copy of that block)
- On a write: invalidate the block in all caches that have the block and reset their bits in directory entry for the block
- Have an "exclusive bit" associated with each block in each cache (so that the cache can update the exclusive block silently)
 local caches

-- Excl bit present in cache.

Lecture on Cache Coherence

Current cache coherence protocols use both



Directory based ?

Snoop based approach for Cache coherence.

Computer Architecture - Lecture 21: Cache Coherence (ETH Zürich, Fall 2020)

1,419 views • Dec 4, 2020

27 0 SHARE SAVE ...



Onur Mutlu Lectures
16.3K subscribers

ANALYTICS

EDIT VIDEO

Lecture on Memory Ordering & Consistency

Diagram: Interconnection Network

Processors: P_1 (red), P_2 (blue)

Shared Memory: F_1 (red), F_2 (blue)

Timeline:

- 0:**
 - P_1 : A (Set $F_1=1$)
 - P_2 : X (Set $F_2=1$)
- 1:**
 - P_1 : B (test $F_2=0$)
 - P_2 : Y (test $F_1=0$)
- 50:**
 - Mem. sends $F_2(0)$ to P_1
 - Mem. sends $F_1(0)$ to P_2
- 51:**
 - P_1 is in Critical Section()
 - P_2 is in Critical Section()
- 100:**
 - Mem. completes A (Too Late!)
 - Mem. completes X (Too Late!)

For P_1 :
A appeared to happen before X

For P_2 :
X appeared to happen before A

Views:

- P_1 's VIEW: $A \rightarrow B \rightarrow X$
- P_2 's VIEW: $X \rightarrow Y \rightarrow A$

Conclusion:
Both CANNOT BE CORRECT!
(from memory's perspective)

Computer Architecture - Lecture 20: Memory Ordering (Memory Consistency) (ETH Zürich, Fall 2020)

976 views • Dec 4, 2020

22 0 SHARE SAVE ...



Onur Mutlu Lectures
16.5K subscribers

ANALYTICS EDIT VIDEO

<https://www.youtube.com/watch?v=Suy09mzTbiQ&list=PL5Q2soXY2Zi9xidylgBxUz7xRPS-wisBN&index=37>

Lecture on Cache Coherence & Consistency

■ Computer Architecture, Fall 2020, Lecture 21

- ❑ Cache Coherence (ETH, Fall 2020)
- ❑ <https://www.youtube.com/watch?v=T9WlyezeaII&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=38>

■ Computer Architecture, Fall 2020, Lecture 20

- ❑ Memory Ordering & Consistency (ETH, Fall 2020)
- ❑ <https://www.youtube.com/watch?v=Suy09mzTbiQ&list=PL5Q2soXY2Zi9xidyIgBxUz7xRPS-wisBN&index=37>

■ Computer Architecture, Spring 2015, Lecture 28

- ❑ Memory Consistency & Cache Coherence (CMU, Spring 2015)
- ❑ <https://www.youtube.com/watch?v=JfjT1a0vi4E&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=32>

■ Computer Architecture, Spring 2015, Lecture 29

- ❑ Cache Coherence (CMU, Spring 2015)
- ❑ <https://www.youtube.com/watch?v=X6DZchnMYcw&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=33>

Digital Design & Computer Arch.

Lecture 23b: Multi-Core Issues in Caching

Prof. Onur Mutlu

ETH Zürich

Spring 2025

22 May 2025